

Villanova Documentation

Table of contents

1 Villanova Technical Documentation	4
I Architecture	5
2 Architecture Overview	6
II Platform	7
3 Entando	8
III Catalog	9
4 Component Catalog	10
5 M2 - Video Anonymization	11
5.1 Purpose	11
5.2 Inputs	11
5.3 Outputs	11
IV How-To	12
6 Configure GPU Support in a Target Environment	13
6.1 Prerequisites	13
6.2 1. Verify GPU Support on the Host	13
6.3 2. Verify the NVIDIA Runtime in K3s	14
6.4 3. Set the Default Runtime in K3s	14
6.5 4. Configure CDI Classes and Containerd Runtime	15
6.6 5. Create the NVIDIA RuntimeClass	15
6.7 6. Install the NVIDIA GPU Operator	16
6.8 7. Install the NVIDIA Device Plugin	17
6.9 8. Validate GPU Scheduling	17

6.10	Troubleshooting	17
6.10.1	Error: <code>Unknown runtime handler nvidia</code>	17
6.10.2	GPU Operator does not become ready	18
6.10.3	Reinstall the operator	18
7	How-To: Create Blueprint	19
8	How-To: Create a Target Environment in Villanova Platform	20
8.1	Assumptions	20
8.2	Prerequisites	20
8.3	1. MetalLB	20
8.3.1	Install MetalLB (only if not configured)	21
8.3.2	Configure the IP address pool	21
8.4	2. DNS	21
8.4.1	Cloudflare Configuration	22
8.5	3. Manage TLS Certificates with cert-manager	22
8.5.1	Install Cert-Manager (if not installed)	22
8.5.2	Create a ClusterIssuer for Let's Encrypt	22
8.6	4. Create the Target Namespace	24
8.7	5. Configure Service Accounts and Permissions	24
8.7.1	Create service account for operations API	24
8.7.2	Create service account for Villanova workloads	25
8.8	6. Create the Target Environment from the UI	26

1 Villanova Technical Documentation

Benvenuti nella documentazione tecnica di Villanova.

Part I

Architecture

2 Architecture Overview

Part II

Platform

3 Entando

Part III

Catalog

4 Component Catalog

5 M2 - Video Anonymization

5.1 Purpose

5.2 Inputs

5.3 Outputs

Part IV

How-To

6 Configure GPU Support in a Target Environment

This guide explains how to validate NVIDIA GPU support on a K3s target environment and how to enable GPU workloads for Villanova deployments.

6.1 Prerequisites

- A K3s target environment already configured for Villanova.
- An NVIDIA GPU installed on at least one node.
- NVIDIA drivers already installed on the host.
- `kubectl` access to the cluster.
- `helm` installed on the administration host.

6.2 1. Verify GPU Support on the Host

Check that the node sees the GPU:

```
nvidia-smi
```

Expected result: the command prints the GPU model, driver version, and CUDA version without errors.

Verify that the NVIDIA container toolkit is available:

```
nvidia-container-toolkit --version
```

Expected result: the command prints the installed toolkit version.

6.3 2. Verify the NVIDIA Runtime in K3s

Check that the NVIDIA runtime is registered in the active containerd configuration:

```
sudo grep -A3 "nvidia" /var/lib/rancher/k3s/agent/etc/containerd/config.toml
```

Expected result:

```
[plugins.'io.containerd.cri.v1.runtime'.containerd.runtimes.'nvidia']  
[plugins.'io.containerd.cri.v1.runtime'.containerd.runtimes.'nvidia'.options]  
  BinaryName = "/usr/bin/nvidia-container-runtime"
```

If the runtime is missing, also inspect the K3s containerd template:

```
sudo cat /var/lib/rancher/k3s/agent/etc/containerd/config.toml.tpl  
sudo ls -la /etc/containerd/conf.d/
```

On K3s installations that use `imports = ["/etc/containerd/conf.d/*.toml"]`, an empty `/etc/containerd/conf.d/` directory can prevent the NVIDIA runtime from being configured as expected.

6.4 3. Set the Default Runtime in K3s

Update the K3s configuration file:

```
write-kubeconfig-mode: "0600"  
disable:  
  - servicelb  
tls-san:  
  - "<PUBLIC_IP>"  
node-name: "<NODE_NAME>"  
default-runtime: nvidia
```

File: `/etc/rancher/k3s/config.yaml`

Restart K3s after the change.

6.5 4. Configure CDI Classes and Containerd Runtime

Create the CDI directory and generate both workload and management CDI classes:

```
sudo mkdir -p /etc/cdi

# Workload class
sudo nvidia-ctk cdi generate --output=/etc/cdi/nvidia.yaml

# Management class (used internally by the device plugin)
sudo nvidia-ctk cdi generate --vendor=management.nvidia.com --class=gpu --output=/etc/cdi/management.yaml
```

Verify both classes are present:

```
sudo nvidia-ctk cdi list
```

Expected result: entries for both `nvidia.com/gpu=*` and `management.nvidia.com/gpu=*` are listed.

Ensure the NVIDIA runtime stanza exists in the K3s containerd template:

```
sudo cp /var/lib/rancher/k3s/agent/etc/containerd/config.toml \
  /var/lib/rancher/k3s/agent/etc/containerd/config.toml.tpl

sudo tee -a /var/lib/rancher/k3s/agent/etc/containerd/config.toml.tpl > /dev/null <<'EOF'
[plugins."io.containerd.grpc.v1.cri".containerd.runtimes.nvidia]
runtime_type = "io.containerd.runc.v2"
[plugins."io.containerd.grpc.v1.cri".containerd.runtimes.nvidia.options]
BinaryName = "/usr/bin/nvidia-container-runtime"
EOF
```

Restart K3s:

```
sudo systemctl restart k3s
```

6.6 5. Create the NVIDIA RuntimeClass

Apply the runtime class used by GPU workloads:

```
kubectl apply -f - <<'EOF'
apiVersion: node.k8s.io/v1
kind: RuntimeClass
metadata:
  name: nvidia
handler: nvidia
EOF
```

Verify that it exists:

```
kubectl get runtimeclass
```

Expected result: a RuntimeClass named `nvidia` is listed.

6.7 6. Install the NVIDIA GPU Operator

Add the NVIDIA Helm repository and update the index:

```
helm repo add nvidia https://helm.ngc.nvidia.com/nvidia
helm repo update
```

Install the operator with K3s-friendly settings:

```
helm install --wait --generate-name \
  -n gpu-operator --create-namespace \
  nvidia/gpu-operator \
  --version=v26.3.3 \
  --set driver.enabled=false \
  --set cdi.enabled=true \
  --set cdi.default=true \
  --set toolkit.env[0].name=CONTAINERD_RUNTIME_CLASS \
  --set toolkit.env[0].value=nvidia
```

This configuration assumes the NVIDIA driver is already managed on the host and only the Kubernetes GPU components need to be deployed.

6.8 7. Install the NVIDIA Device Plugin

If you are not using the full GPU Operator stack, install the NVIDIA device plugin directly:

```
helm repo add nvdp https://nvidia.github.io/k8s-device-plugin
helm repo update
helm install nvdp nvdp/nvidia-device-plugin --namespace kube-system --create-namespace
```

6.9 8. Validate GPU Scheduling

Run a test workload:

```
kubectl run gpu-test --rm -it --restart=Never \
  --image=nvidia/cuda:12.4.0-base-ubuntu22.04 \
  --overrides='{"spec":{"runtimeClassName":"nvidia"}}' \
  -- nvidia-smi
```

Expected result: the pod starts successfully and prints `nvidia-smi` output from inside the container.

You can also validate allocatable GPU resources:

```
kubectl get nodes -o json | jq '.items[].status.allocatable'
```

Expected result: at least one node exposes `nvidia.com/gpu` in its allocatable resources.

To specifically check for the GPU allocatable key:

```
kubectl get nodes -o json | jq '.items[].status.allocatable | select(. "nvidia.com/gpu" != null)'
```

6.10 Troubleshooting

6.10.1 Error: Unknown runtime handler nvidia

Check these items:

- `default-runtime`: `nvidia` is present in `/etc/rancher/k3s/config.yaml`.
- The `RuntimeClass` named `nvidia` exists.
- The NVIDIA runtime is present in `/var/lib/rancher/k3s/agent/etc/containerd/config.toml`.
- `/etc/containerd/conf.d/` is not empty if the K3s template imports configuration from that path.

6.10.2 GPU Operator does not become ready

If the operator gets stuck on `dcgm-exporter`, disable DCGM components:

```
kubectl patch clusterpolicy cluster-policy --type merge \
  -p '{"spec":{"dcgm":{"enabled":false},"dcgmExporter":{"enabled":false}}}'
```

Then re-check the operator pods:

```
kubectl get pods -n gpu-operator
```

6.10.3 Reinstall the operator

If you need a clean reinstall:

```
helm uninstall -n gpu-operator $(helm list -n gpu-operator -q)
kubectl delete pods -n gpu-operator --all --grace-period=0 --force
```

Re-run the install command after cleanup.

Note

Keep API tokens, cloud credentials, and other secrets out of the documentation. Use placeholders such as `<CLOUDFLARE_API_TOKEN>` in examples.

7 How-To: Create Blueprint

8 How-To: Create a Target Environment in Villanova Platform

This guide explains how to prepare a Kubernetes target environment for Villanova workloads.

8.1 Assumptions

A target environment is a Kubernetes cluster that is reachable from the Villanova Platform. The target environment must have a public IP address and a wildcard DNS record for the domain used by the workloads.

8.2 Prerequisites

- An existing K3s cluster provisioned with Ansible.
- Traefik installed as ingress controller.
- `kubect1` configured to access the cluster.
- Access to DNS configuration for your domain.

8.3 1. MetalLB

MetaLB is a load-balancer implementation for bare metal Kubernetes clusters, using standard routing protocols. It allows you to assign external IP addresses to services of type `LoadBalancer`. In Villanova deployments, Traefik is configured as a `LoadBalancer` service. To expose the service to the internet, you need to configure MetalLB in your target environment. MetalLB assigns an external IP to the Traefik `LoadBalancer` service.

Verify that Traefik gets an external IP:

```
kubect1 get svc -A
```

Expected result: the `kube-system/traefik` service has type `LoadBalancer` with an `EXTERNAL-IP`.

8.3.1 Install MetalLB (only if not configured)

```
kubectl apply -f https://raw.githubusercontent.com/metallb/metallb/v0.16.1/config/manifests/
```

8.3.2 Configure the IP address pool

Create a file named `metallb-config.yaml`:

```
apiVersion: metallb.io/v1beta1
kind: IPAddressPool
metadata:
  name: production-pool
  namespace: metallb-system
spec:
  addresses:
    - 93.119.169.188/32
---
apiVersion: metallb.io/v1beta1
kind: L2Advertisement
metadata:
  name: l2-advert
  namespace: metallb-system
spec:
  ipAddressPools:
    - production-pool
```

Apply the configuration:

```
kubectl apply -f metallb-config.yaml
```

8.4 2. DNS

Target Environments can have a wildcard DNS record pointing to the Traefik external IP. This allows Villanova applications to be accessed via subdomains of your domain. Each DNS provider has its own procedure for creating a wildcard DNS record. Consult your provider's documentation for instructions, or use the following example for Cloudflare: Create a wildcard DNS record pointing to the Traefik external IP.

8.4.1 Cloudflare Configuration

- Log in to your Cloudflare account.
- Select the domain you want to configure.
- Go to the DNS settings.
- Add a new record:
- Type: A
- Name: *
- IPv4 address: <Traefik External IP>
- TTL: Auto

! Important

Unflag Proxy if select, use DNS only

Example:

- *.edge1.villanova.ai -> 93.119.169.188

Quick test:

- Open <https://test.edge1.villanova.ai/> in a browser.

8.5 3. Manage TLS Certificates with cert-manager

Check if cert-manager is installed:

```
kubectl get pods -n cert-manager
```

8.5.1 Install Cert-Manager (if not installed)

```
kubectl apply -f https://github.com/cert-manager/cert-manager/releases/download/v1.16.2/cert-
```

8.5.2 Create a ClusterIssuer for Let's Encrypt

8.5.2.1 Version 1: Use DNS-01 challenge (recommended)

Create a token in Cloudflare with permissions to manage DNS records. Set Then create a Kubernetes secret with the token:

```
kubectl create secret generic cloudflare-api-token-secret \
  --from-literal=api-token='<CLOUDFLARE_API_TOKEN>' \
  --namespace cert-manager
```

Apply the configuration:

```
apiVersion: cert-manager.io/v1
kind: ClusterIssuer
metadata:
  name: letsencrypt-prod
spec:
  acme:
    server: https://acme-v02.api.letsencrypt.org/directory
    email: snicotra@villanova.ai
    privateKeySecretRef:
      name: letsencrypt-prod-key
    solvers:
      - dns01:
          cloudflare:
            apiTokenSecretRef:
              name: cloudflare-api-token-secret
              key: api-token
          selector:
            dnsZones:
              - "villanova.ai"
```

```
kubectl apply -f letsencrypt.yaml
```

8.5.2.2 Version 2: Use HTTP-01 challenge (default)

Create a ClusterIssuer for Let's Encrypt in a file named letsencrypt.yaml:

```
apiVersion: cert-manager.io/v1
kind: ClusterIssuer
metadata:
  name: letsencrypt-prod
spec:
  acme:
    server: https://acme-v02.api.letsencrypt.org/directory
    email: tuamail@esempio.com
```

```

privateKeySecretRef:
  name: letsencrypt-prod-key
solvers:
  - dns01:
      cloudflare:
        apiTokenSecretRef:
          name: cloudflare-api-token-secret
          key: api-token
      selector:
        dnsZones:
          - "villanova.ai"

```

Apply it:

```
kubectl apply -f letsencrypt.yaml
```

Verify

```
kubectl get secrets -n cert-manager
```

8.6 4. Create the Target Namespace

In the target environment, create a namespace for Villanova workloads:

```
kubectl create namespace villanova-edge
```

8.7 5. Configure Service Accounts and Permissions

8.7.1 Create service account for operations API

```

kubectl create serviceaccount operation-api -n default
kubectl create clusterrolebinding operation-api-sa-admin \
  --clusterrole=cluster-admin \
  --serviceaccount=default:operation-api

```

Generate a token:


```
kubectl create token operation-api --duration 200000h
```

i Note

Token will be used in the Villanova UI while creating the target environment.

8.7.2 Create service account for Villanova workloads

```
---
apiVersion: v1
kind: ServiceAccount
metadata:
  name: entando-workload
  namespace: villanova-edge
automountServiceAccountToken: true
---
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: rollout-reader
  namespace: villanova-edge
rules:
- apiGroups: ["apps"]
  resources: ["deployments", "replicasets","statefulsets"]
  verbs: ["get", "list", "watch"]
- apiGroups: [""]
  resources: ["pods"]
  verbs: ["get", "list", "watch"]
---
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: secret-manager
  namespace: villanova-edge
rules:
- apiGroups: [""]
  resources: ["secrets"]
  verbs: ["get", "create", "update", "patch", "list"]
---
apiVersion: rbac.authorization.k8s.io/v1
```

```

kind: RoleBinding
metadata:
  name: rollout-reader-binding
  namespace: villanova-edge
subjects:
- kind: ServiceAccount
  name: entando-workload
  namespace: villanova-edge
roleRef:
  apiGroup: rbac.authorization.k8s.io
  kind: Role
  name: rollout-reader
---
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: secret-manager-binding
  namespace: villanova-edge
subjects:
- kind: ServiceAccount
  name: entando-workload
  namespace: villanova-edge
roleRef:
  apiGroup: rbac.authorization.k8s.io
  kind: Role
  name: secret-manager

```

Apply the configuration:

```
kubectl apply -f entando-workload.yaml
```

8.8 6. Create the Target Environment from the UI

After infrastructure setup is complete, create the target environment from the Villanova UI.

Use these values:

- Name: Villanova Edge 1
- Environment type: production
- Cluster Address: (https://<external-ip>:6443)
- Namespace (villanova-edge)

- Service account token (`operation-api`)

in Advanced settings, set: - Domain (for example `edge1.villanova.ai`)

If connection is successful, the target environment will be created and ready to deploy workloads.